

Сопроводительная документация проекта «СтройКонтроль»

Проект: Автоматизированный сервис поиска нарушений на строительных площадках Москвы
Конкурс: «Лидеры цифровой трансформации» (ЛЦТ) 2026, кейс 07
Заказчик: Департамент градостроительной политики города Москвы
Целевая ветка репозитория: main

1. Модель данных

Хранение данных организовано по архитектурному паттерну **Database-per-Service**: каждый микросервис с состоянием владеет собственной физически изолированной базой данных в СУБД PostgreSQL 16. Прямые межсервисные SQL-запросы и внешние ключи (Foreign Keys) между базами исключены; обмен данными между сервисами осуществляется исключительно по протоколу HTTP REST API с передачей сквозных идентификаторов в формате UUIDv4.

1.1. Базы данных микросервисов

Сервис	База данных	Назначение и состав хранимых сущностей
Сервис плана (plan-service)	plandb	Нормативный план и правила: строительные объекты (object), календарные графики производства работ (stage), справочник видов работ (work_type), технологические правила соответствия «этап → техника» (stage_rule), рабочий производственный календарь (work_calendar).
Сервис площадки (site-service)	sitedb	Физическая площадка и наблюдения: реестр стационарных камер (camera), разметка полигональных участков и опасных зон (zone), каталог принятых снимков (image), результаты распознавания техники на кадрах (detection), 30-минутные окна агрегации наблюдений (observation_window), статус видимости зон (area_visibility), оценки визуальной стадии объекта (stage_observation).
Сервис анализа (analysis-service)	analysisdb	Сверка план-факт и инциденты: прогоны план-факт анализа (analysis_run), журнал зафиксированных отклонений и нарушений (deviation), реестр настраиваемых правил отклонений (deviation_rule), фактические показатели прогресса и прогноз завершения этапов (stage_fact), ежедневные метрики активности (daily_activity), учет использования техники (equipment_usage), сгенерированные аналитические PDF-отчеты (report).
Объектное хранилище (gateway / SeaweedFS)	S3 Buckets	Бинарные медиафайлы: оригиналы и оптимизированные снимки с камер (бакет images), сформированные PDF-отчеты с подтверждающими материалами (бакет reports).

1.2. Хранение информации о строительной технике

Учет техники в системе охватывает весь жизненный цикл — от сырой детекции на кадре до расчета эффективного рабочего времени:

- Единый классификатор техники (packages/contracts/equipment_classes.yaml):**
Централизованный файл-контракт для всех сервисов системы. Содержит 16 классов строительной техники (8 базовых классов технического задания + 8 дополнительных классов), текстовые англоязычные промпты для открытого словаря детектора, признак транзитности (transient), признак работы без перемещения (works_in_place), а также таблицу соответствия меток внешних датасетов (aliases).
- Сырые детекции на снимках (sitedb.detection):**
Каждая обнаруженная единица техники сохраняется со следующими параметрами:
 - image_id — идентификатор снимка;
 - equipment_class — нормализованный код класса техники;
 - bbox — нормализованные координаты ограничивающей рамки [y_min, x_min, y_max, x_max] в диапазоне от 0.0 до 1.0;
 - bottom_center — точка контакта машины с землей [x, y] для точной геометрической привязки к полигонам зон;
 - zone_id — идентификатор полигона зоны, в границы которого попала точка контакта;
 - confidence — степень уверенности детектора (число от 0.0 до 1.0);
 - model_version — имя и версия модели нейросети, выполнившей распознавание.
- Агрегированный учет работы и простоев (analysisdb.equipment_usage):**
В рамках 30-минутных окон наблюдений (observation_window) фиксируется статус каждой машины:
 - WORKING — активная работа (перемещение точки контакта свыше порога или стационарная работа кранов и бетононасосов на пятне застройки);

- `IDLE` — необоснованный простой (техника неподвижна несколько окон подряд);
- `OUT_OF_ZONE` — нахождение машины за пределами целевого участка производства работ.

1.3. Хранение информации об этапах работ

Система поддерживает 4 типа объектов капитального строительства (`packages/contracts/enums.yaml`): - `RESIDENTIAL_MONOLITH` — жилые монолитные здания; - `RESIDENTIAL_PANEL` — жилые панельные здания; - `PUBLIC_BUILDING` — общественные здания (школы, детские сады, поликлиники); - `ROAD` — автомобильные дороги и объекты улично-дорожной сети.

Структура данных планирования включает:

- 1. Календарный график СМР (`plandb.stage`):**
Каждая позиция графика включает плановые даты старта и окончания (`plan_start` , `plan_end`), нормативную продолжительность в рабочих днях (`norm_duration_days`), перечень предшествующих этапов и тип технологической связи (`predecessors` : FS — Finish-to-Start, SS — Start-to-Start), признак нахождения на критическом пути (`is_critical_path`), привязку к физическому участку (`area_name`) и текущий статус (`PLANNED` , `IN_PROGRESS` , `COMPLETED` , `CANCELLED`). Для монолитного домостроения реализована автогенерация по МРР-3.2.81-12, для остальных типов объектов график импортируется из таблиц заказчика.
- 2. Правила соответствия «этап → техника» (`plandb.stage_rule`):**
Определяют нормативные требования к механизации каждого вида строительных работ:
 - `required` — перечень обязательных классов техники и их минимальное количество;
 - `allowed` — перечень разрешенной вспомогательной техники;
 - `signature` — технологическая сигнатура (набор машин или визуальная стадия), однозначно свидетельствующая о фактическом старте этапа;
 - `min_sessions` — минимальное число рабочих сессий подтверждения.
- 3. Иерархический классификатор видов работ (`plandb.work_type`):**
Четырехуровневый справочник работ г. Москвы с кодировкой видов строительно-монтажных процессов и указанием их применимости для различных типов объектов капитального строительства.

1.4. Хранение информации об отклонениях и нарушениях

Контроль хода строительства осуществляется на основе **10 автоматизированных правил выявления нарушений и отклонений (D1–D10)**.

Настройки правил хранятся в таблице `analysisdb.deviation_rule` (базовая критичность, числовые пороги, условия временной эскалации и параметрические шаблоны сообщений из конфигурации `services/analysis-service/data/deviation_rules.yaml`). Все 10 правил преднастроены в системе:

Код	Наименование правила	Условие срабатывания	Базовая критичность
D1	Отсутствие обязательной техники	На активном по графику этапе в течение заданного числа сессий отсутствует вся обязательная техника	MEDIUM (эскалация до HIGH через 1 день)
D2	Неполный комплект техники	На активном этапе зафиксирована лишь часть обязательной техники (не соблюден минимальный комплект)	MEDIUM (эскалация до HIGH через 3 дня)
D3	Непредусмотренная техника / опережение	На участке зафиксирована техника, не входящая в правила текущих этапов (при совпадении с будущим этапом фиксируется возможное опережение)	MEDIUM
D4	Необоснованный простой техники	Техника находится на площадке без движения свыше допустимого числа рабочих сессий подряд	LOW (эскалация до MEDIUM через 1 день)
D5	Техника вне целевой рабочей зоны	Машины выполняют операции за пределами разрешенной рабочей зоны объекта	MEDIUM
D6	Нарушение границ опасной зоны	Заезд строительной техники или нахождение людей в границах опасной зоны	HIGH
D7	Несоответствие визуальной стадии	Фактический внешний вид объекта по данным классификатора отстает от стадии, требуемой календарным планом	HIGH
D8	Затягивание сроков этапа	Плановый срок завершения этапа истек, но на площадке продолжают фиксироваться машины его сигнатуры	HIGH
D9	Срыв планового старта работ	Наступила плановая дата старта этапа, но в течение первых 3 рабочих дней технологическая сигнатура работ не обнаружена	HIGH
D10	Участок вне контроля ИИ	Наблюдение невозможно из-за внешних факторов (не размечена зона, загрязнена оптика, темнота, перекрытие обзора)	INFO

Журнал зафиксированных инцидентов сохраняется в `analysisdb.deviation` со ссылками на подтверждающие фотокадры (`evidence_image_ids`), структурированными фактами нарушения в формате JSONB (`facts`), статусом жизненного цикла (`ACTIVE` , `RESOLVED`)

и вердиктом ответственного инженера (`NEW` , `CONFIRMED` , `REJECTED`).

2. Структура кода и комментарии

2.1. Организация монорепозитория

Проект спроектирован в виде модульного монорепозитория со строгим разграничением сфер ответственности:

ConstructionSupervision/

apps/

web/

services/

gateway/

plan-service/

site-service/

vision-service/

analysis-service/

packages/

contracts/

py-common/

ts-api-client/

ml/

scripts/

React 18 SPA (TypeScript): дашборд, интерактивная диаграмма Ганта,
лента отклонений, редактор полигональных зон и фотокамер

Nginx: единая точка входа (Reverse Proxy), раздача веб-интерфейса,
маршрутизация REST API, Swagger UI и объектного хранилища

Сервис плана: нормативный график СМР по МРР-3.2.81-12, WBS-шаблоны,
производственный календарь, технологические правила «этап → техника»

Сервис площадки: камеры, полигоны зон, прием снимков, воркер

распознавания (site-worker на базе arq/Redis), окна наблюдений

Сервис компьютерного зрения: детекция техники и людей (YOLO-World v2),
классификация визуальной стадии (OpenCLIP), алгоритмическая оценка качества

Сервис анализа: правила нарушений D1-D10, расчет прогресса и CPM/SPI,
генерация PDF-отчетов с текстовым резюме локальной нейросетью

Межсервисные спецификации: equipment_classes.yaml, enums.yaml, OpenAPI

Общая Python-библиотека: логирование, аутентификация, обработка ошибок

Автогенерируемый типизированный TypeScript-клиент для веб-интерфейса

Офлайн-модуль ML: подготовка датасетов, дообучение YOLO-World, скрипты
честной оценки качества (mAP50, матрицы ошибок) на тестах

Утилиты развертывания: загрузка весов моделей (fetch_models.py),
предзаполнение демо-данных (seed.py), сквозное тестирование (e2e.py)

2.2. Архитектура микросервисов и изоляция доменной логики

Каждый микросервис бэкенда построен по принципам многослойной чистой архитектуры:

- `src/core/` — **изолированная доменная логика**. Включает чистые детерминированные функции: правила выявления нарушений (D1–D10), расчет критического пути (CPM) и показателей освоенного объема (SPI), геометрическую привязку точек к полигонам зон (Ray Casting), алгоритмическую оценку размытия кадра. Модули ядра не содержат обращений к базам данных, сетевых вызовов или системного ввода-вывода, благодаря чему на 100% покрыты быстрыми модульными тестами.
- `src/dal/` — слой доступа к данным на базе асинхронной ORM SQLAlchemy 2.0 (декларативные модели, репозитории с транзакционными границами).
- `src/api/` — транспортный слой FastAPI: роутеры, сквозная валидация схем запросов и ответов через Pydantic v2, обработчики типовых доменных исключений.
- `src/services/` — прикладные сценарии оркестрации межсервисного взаимодействия и длительных бизнес-процессов.

2.3. Культура комментирования кода

- Исходный код снабжен исчерпывающими комментариями и docstring-описаниями на русском языке.
- Комментарии строго ориентированы на пояснение архитектурных и инженерных причин («почему код спроектирован именно так»), а не на банальный пересказ синтаксиса.
- Все алгоритмические допущения и формулы снабжены прямыми ссылками на нормативные документы (например, нормативные сроки периодов строительства по таблице 1 МРР-3.2.81-12, формулы расчета задержки критического пути CPM).
- Заголовочные блоки модулей содержат указание функционального назначения и соответствия разделам технического задания.

3. Условия, ограничения и качество машинного обучения

3.1. Архитектура нейросетевого конвейера и используемые модели

В системе реализован специализированный многокомпонентный конвейер компьютерного зрения, исключающий взаимные блокировки и обеспечивающий автономность работы:

1. Детектор строительной техники и персонала:
 - Архитектура: Ultralytics YOLO-World v2 (размерность small, yolov8s-worldv2, 35 МБ). Модель открытого словаря (open-vocabulary), в которой целевые классы задаются текстовыми промптами на английском языке из файла packages/contracts/equipment_classes.yaml.
 - Базовые веса: yolov8s-worldv2.pt (предобученные веса Ultralytics).
 - Рабочие веса стенда: yolov8s-worldv2-ce-ulima-v1.pt (ce-ulima-v1), дообученные на объединенном наборе данных строительной техники Construction Equipment и стационарных камерах 4К ЖК «Лима». Дообучение выполняется на самом формате YOLO-World, что сохраняет возможность динамического расширения классов через текстовые промпты без изменения программного кода детектора.
2. Классификатор визуальной стадии строительства:
 - Архитектура: OpenCLIP ViT-B-32 (чекпоинт laion2b_s34b_b79k, файл data/models/openclip-vit-b32/open_clip_model.safetensors, 600 МБ).
 - Принцип работы: zero-shot сопоставление визуального эмбединга всего кадра стройплощадки с усредненными текстовыми эмбедингами стадий строительства (котлован PIT, свайное поле PILES, фундамент FOUNDATION, монолитный каркас FRAME, фасад и кровля FACADE, благоустройство LANDSCAPING) из конфигурации services/vision-service/data/stage_prompts.yaml.
3. Текстовый энкодер промптов техники:
 - Модель: OpenAI CLIP ViT-B/32 (data/models/clip/ViT-B-32.pt, 340 МБ). Локально преобразует текстовые описания классов техники в векторные представления для YOLO-World при старте контейнера, устраняя потребность в доступе к интернету во время работы.
4. Алгоритмический модуль контроля пригодности снимков:
 - Анализ качества (services/vision-service/src/core/quality.py) выполняется мгновенно классическими математическими методами: расчет интегральной освещенности кадра в градациях серого и оценка степени расфокусировки по дисперсии оператора Лапласа (Laplacian variance), приведенной к калиброванной шкале. Позволяет отсекать ночные, засвеченные или загрязненные кадры без расхода ресурсов GPU.
5. Локальная языковая модель аналитического резюме:
 - Модель: Gemma 4 E4B Instruct в формате квантования GGUF Q4_K_M (файл data/models/llm/gemma-4-E4B-it-Q4_K_M.gguf, 4.9 ГБ), развернутая в микросервисе llm на движке llama.cpp (серверный образ с аппаратным ускорением CUDA). Модель принимает на вход структурированные JSON-факты прогона анализа и формирует связный текст для руководителя проекта. При отсутствии ресурсов GPU или недоступности модели сервис автоматически генерирует резюме по детерминированному инженерному шаблону.

3.2. Данные обучения, валидация и объективные метрики

В проекте принят принцип абсолютной объективности метрик: результаты оцениваются строго на независимых тестовых выборках, а все ограничения моделей открыто фиксируются.

Набор данных	Объем и характеристики	Роль в проекте
Construction Equipment (Kaggle)	20 000 кадров (16 754 train, 3 228 val), неподвижные камеры наружного наблюдения, осень/зима/ночь, 17 исходных классов техники.	Обучающая выборка детектора техники. Метки классов нормализованы через синонимы в equipment_classes.yaml.
Universidad de Lima (Перу, CC BY 4.0)	1 046 кадров 4К (3840×2160) с 4 стационарных камер реальной стройки за 4 месяца, разметка 8 классов техники и людей.	Разбит по независимым съемочным сериям: 720 кадров в обучении, 62 в валидации, 264 в отложенном тесте. Источник демо-хронологии.
Тестовый набор экспертов (1ct-test)	100 разнородных снимков организаторов хакатона, около 10 различных стройплощадок Москвы, лето и зима, 600 единиц строительной техники.	Независимая тестовая выборка с верифицированной ручной разметкой в LabelMe. Полностью исключена из обучения.

Результаты измерений качества детекции (mAP50):

- На отложенном тесте площадки Лима (264 кадра 4К):
- Базовая модель zero-shot YOLO-World v2 без дообучения: **mAP50 = 0.07** (модель путает бытовки и контейнеры с самосвалами, генерируя в среднем 8.6 ложных рамок на кадр).

- Дообученная модель `se-ulima-v1`: **mAP50 = 0.81** (по классам: бетоносмесители — 0.99, автобетононасосы — 0.99, самосвалы — 0.92, башенные краны — 0.87, погрузчики — 0.84; среднее число ложных детекций снижено до 0.1 на кадр).
- **На контрольной выборке организаторов (100 сложных кадров, 600 машин):**
- Базовая zero-shot модель дает колоссальное число ложных срабатываний (**521 ложная рамка** на 100 кадров при пороге 0.35, называя столбы и конструкции сваебойными установками и кранами).
- Дообученная модель `se-ulima-v1` успешно подавляет ложные детекции (**всего 18 ложных рамок** против 521 у базовой модели).
- При рабочем пороге 0.35 модель уверенно находит массовую технику, присутствовавшую в обучающих датасетах (экскаваторы — 68 верных детекций, самосвалы), обеспечивая точность Precision 0.73 по экскаваторам.
- Ограничение: техника, специфичная для московскихстроек и отсутствующая в открытых датасетах с открытой лицензией (буровые сваебойные установки с тяжелыми мачтами, специализированные гусеничные краны), дообученной моделью пропускается (mAP50 = 0.00 по буровым). Это объективное инженерное ограничение текущих открытых данных, для преодоления которого система переводит нераспознанные участки в статус D10 для ручной проверки инженером.

Быстродействие конвейера компьютерного зрения:

- **Инференс на GPU (NVIDIA RTX 3060 Laptop):** медиана **29 мс**, 90-й перцентиль **58 мс** на снимках организаторов; **23 мс / 53 мс** на кадрах 4K (требование технического задания: ≤ 100 мс выполняется с двукратным запасом).
- **Инференс на CPU (входное разрешение 960px):** медиана **900 мс**, 90-й перцентиль **985 мс** (требование технического задания: $\leq 1-2$ с полностью соблюдено).

3.3. Распознаваемые типы техники

В соответствии с техническим заданием и контрактом `equipment_classes.yaml` система классифицирует 16 типов строительной техники и ресурсов:

- 1. **8 обязательных базовых классов по ТЗ:**
 - Самосвал (`dump_truck`)
 - Экскаватор (`excavator`)
 - Дорожный каток (`roller`)
 - Кран-манипулятор (`manipulator_crane`)
 - Автобетоносмеситель (`concrete_mixer`)
 - Бульдозер (`bulldozer`)
 - Бортовой грузовик / тягач (`truck`)
 - Автокран мобильный (`truck_crane`)
- 2. **8 расширенных классов системы:**
 - Башенный кран (`tower_crane`) — ключевой маркер возведения надземного монолитного каркаса
 - Автобетононасос (`concrete_pump`) — маркер заливки фундаментной плиты и перекрытий
 - Буровая / сваебойная установка (`pile_driver`) — маркер устройства свайного поля
 - Фронтальный / мини-погрузчик (`loader`)
 - Асфальтоукладчик (`asphalt_paver`)
 - Автогрейдер (`grader`)
 - Малотоннажный грузовик / фургон (`light_truck`)
 - Рабочий персонал / строители (`person`) — для выявления людей в опасных зонах

3.4. Поддерживаемые типы объектов и технологические этапы СМР

Система поддерживает 4 канонических типа объектов капитального строительства (`packages/contracts/enums.yaml`): 1. **Монолитное домостроение (RESIDENTIAL_MONOLITH)**: преднастроен нормативный расчет сроков по МРР-3.2.81-12 и WBS-шаблон из 12 этапов наружных СМР. 2. **Панельное домостроение (RESIDENTIAL_PANEL)**: графики с контролем монтажной техники (башенные и монтажные краны, панелевозы, длинномеры). 3. **Общественные здания (PUBLIC_BUILDING)**: школы, детские сады, поликлиники, административные комплексы. 4. **Дорожное строительство и благоустройство (ROAD)**: контроль дорожной техники (асфальтоукладчики, катки, грейдеры, самосвалы).

Для панельных домов, общественных зданий и дорог график производства работ импортируется из файлов Excel/таблиц заказчика (`POST /api/v1/plan/objects/{id}/schedule/import`) с автоматическим сопоставлением со Сводным классификатором строительных работ г. Москвы (`plandb.work_type`). Все 10 правил контроля нарушений (D1–D10) применяются универсально ко всем типам объектов.

Ниже приведена эталонная технологическая карта наружных этапов строительства (на примере шаблона `services/plan-service/data/wbs_templates.json` и `data/seed/rules.json` по нормативу МРР-3.2.81-12 и таблице 5.2 ТЗ):

Этап (код и наименование)	Фаза работ	Обязательная техника	Допустимая техника	Сигнатура старта / Зона
10.4–10.8 Подготовка территории (снос, вырубка, вывоз мусора)	Подготовительная	Экскаватор (≥ 1 ед.), Самосвал (≥ 1 ед.)	Бульдозер, Погрузчик, Автокран	Экскаватор + Самосвал Пятно застройки (BUILDING_FOOTPRINT)
10.11 Обустройство строительной площадки	Подготовительная	Кран-манипулятор или Грузовик (≥ 1 ед.)	Автокран, Самосвал, Дорожный каток	Кран-манипулятор Периметр площадки (PERIMETER)
12.3.2 Ограждение котлована, устройство свайного поля	Подземная часть	Буровая/сваебойная установка (≥ 1 ед.)	Автокран, Бетоносмеситель, Бетононасос, Экскаватор	Буровая установка (стадия PILES) Котлован (PIT)
12.3.1 Разработка грунта котлована	Подземная часть	Экскаватор (≥ 1 ед.), Самосвал (≥ 2 ед.)	Бульдозер, Погрузчик	Экскаватор + Самосвалы (стадия PIT) Котлован (PIT)
12.3.3 Устройство основания (песок, щебень, подготовка)	Подземная часть	Самосвал (≥ 1 ед.), Каток или Бульдозер (≥ 1 ед.)	Погрузчик, Автобетоносмеситель	Дорожный каток (стадия PIT) Котлован (PIT)
монтаж-крана Монтаж башенного крана	Подземная часть	Автокран (≥ 1 ед.)	Бортовой грузовик	Автокран + Башенный кран Пятно застройки (BUILDING_FOOTPRINT)
12.3.4 Фундаментная плита, монолит ниже 0	Подземная часть	Автобетоносмеситель (≥ 1 ед.), Бетононасос (≥ 1 ед.)	Башенный кран, Автокран, Бортовой грузовик	Бетоносмеситель + Бетононасос (стадия FOUNDATION) Котлован (PIT)
12.3.10 Гидроизоляция стен, обратная засыпка пазух	Подземная часть	Экскаватор или Погрузчик (≥ 1 ед.), Самосвал (≥ 1 ед.)	Дорожный каток	Самосвал (стадия FOUNDATION) Котлован (PIT)
12.4.4 / 12.4.10 Надземный каркас, бетонирование колонн	Надземная часть	Башенный кран (≥ 1 ед.) [12.4.10: Миксер + Бетононасос]	Бетоносмеситель, Бетононасос, Грузовик	Башенный кран (стадия FRAME) Пятно застройки (BUILDING_FOOTPRINT)
12.4.8 Наружные стены, кровля, фасадные конструкции	Ограждающие конструкции	Башенный кран, Автокран или Манипулятор (≥ 1 ед.)	Бортовой грузовик	Визуальная стадия FACADE Пятно застройки (BUILDING_FOOTPRINT)
12.5.1 Прокладка наружных инженерных сетей	Инженерные сети	Экскаватор (≥ 1 ед.)	Самосвал, Кран-манипулятор, Дорожный каток	Экскаватор Периметр площадки (PERIMETER)
12.7 Благоустройство территории и дорожные покрытия	Благоустройство	Самосвал (≥ 1 ед.), Дорожный каток (≥ 1 ед.)	Асфальтоукладчик, Погрузчик, Бульдозер	Дорожный каток (стадия LANDSCAPING) Периметр площадки (PERIMETER)

Параметры любого технологического правила (минимальный состав машин, сигнатура старта, допустимые участки) могут быть скорректированы или добавлены через веб-интерфейс без перезапуска сервисов.

3.5. Ограничения системы

- Неблагоприятные погодные условия и оптические помехи:** Кадры, снятые в условиях нулевой видимости (ночь без прожекторов), плотного тумана или сильного загрязнения защитного стекла камеры, автоматически идентифицируются модулем качества (services/vision-service/src/core/quality.py). Данные участки переводятся в статус D10 («Вне контроля ИИ»), блокируя появление ложных отклонений и уведомляя оператора о необходимости проверки.
- Обезличивание и защита персональных данных:** В соответствии с требованиями законодательства РФ система не осуществляет распознавание лиц граждан и государственных регистрационных знаков транспортных средств. В базу данных сохраняются исключительно обезличенные категории техники и координаты ограничивающих прямоугольников.

4. Инструкция по сборке, установке и запуску решения

Решение разворачивается в изолированных Docker-контейнерах под управлением Docker Compose.

4.1. Системные требования

- **Операционная система:** Linux (Ubuntu 22.04 LTS и новее) либо Windows 10/11 с установленным Docker Desktop (режим WSL2).
- **Процессор и память:** не менее 4 процессорных ядер, от 16 ГБ оперативной памяти (ОЗУ).
- **Дисковое пространство:** не менее 35 ГБ свободного пространства на SSD-накопителе (для размещения образов контейнеров, весов нейросетей, базы данных и демо-снимков).
- **Графический ускоритель (опционально):** видеокарта NVIDIA (от 6 ГБ VRAM) с установленным NVIDIA Container Toolkit для аппаратного ускорения инференса.

4.2. Пошаговый запуск (быстрый старт)

Шаг 1. Клонирование репозитория и создание файла конфигурации

```
git clone <URL_РЕПОЗИТОРИЯ>
cd ConstructionSupervision
cp .env.example .env
```

(Для командной строки Windows PowerShell: `copy .env.example .env`)

Шаг 2. Загрузка весов моделей и демонстрационных данных

Скрипт `scripts/fetch_models.py` выполняет потоковую загрузку недостающих бинарных компонентов со сверкой контрольных сумм SHA-256. Загрузка производится во временные файлы `.part`, предотвращая появление поврежденных данных при обрыве соединения:

```
docker compose run --rm tools python scripts/fetch_models.py
```

(При наличии локального интерпретатора Python: `python scripts/fetch_models.py`)

Что именно скачивает скрипт:

- **Группа `models` (базовые веса vision-service в `data/models/`):**
 - `yolov8s-worl1dv2.pt` (35 МБ) — базовая модель детекции техники Ultralytics YOLO-World v2;
 - `openclip-vit-b32/open_clip_model.safetensors` (600 МБ) — модель OpenCLIP ViT-B-32 для классификации визуальной стадии объекта;
 - `clip/ViT-B-32.pt` (340 МБ) — текстовый энкодер OpenAI CLIP для кодирования словаря промптов детектора.
- **Группа `demo` (ассеты демонстрационного стенда):**
 - `yolov8s-worl1dv2-ce-ulima-v1.pt` (35 МБ) — дообученный детектор техники на строительных датасетах (указан в `VISION_DET_WEIGHTS`);
 - `demo-images.zip` (56 хронологических кадров 4K стационарных камер площадки Лима, распаковываются в `data/seed/images/`).
- **Группа `llm` (локальная языковая модель):**
 - `gemma-4-E4B-it-Q4_K_M.gguf` (4.9 ГБ) — квантованная модель Gemma 4 E4B instruct для контейнера `llm` (`llama.cpp`).

Полезные флаги скрипта:

- `--skip llm` — не скачивать модель Gemma (экономия 5 ГБ трафика, если резюме формируется шаблонизатором или внешним API);
- `--skip demo` — не скачивать демонстрационные кадры и дообученные веса;
- `--force` — принудительно перезакачать все файлы заново.

Шаг 3. Запуск контейнеров приложения

- **Стандартный запуск в режиме CPU (подходит для любых систем):**

```
bash docker compose up -d
```

- **Запуск с аппаратным ускорением GPU (видеокарта NVIDIA):**

```
bash docker compose -f docker-compose.yml -f docker-compose.gpu.yml up -d
```

Шаг 4. Проверка готовности сервисов

Скрипт проверяет эндпоинты `/health/ready` всех микросервисов и сводный статус шлюза:

```
docker compose run --rm tools python scripts/health.py
```

Все компоненты (`postgres`, `s3`, `redis`, `plan-service`, `site-service`, `site-worker`, `vision-service`, `analysis-service`, `gateway`) должны вернуть код готовности `UP` (код возврата 0).

Шаг 5. Загрузка демонстрационного сценария

Скрипт инициализирует демонстрационный объект капстроительства, импортирует нормативный график СМР, настраивает геометрию рабочих зон камер и загружает хронологию снимков за три последовательных дня:

```
docker compose run --rm tools python scripts/seed.py
```

Шаг 6. Сквозная верификация системы (E2E-тест)

Запуск автоматического теста сквозного контура:

```
docker compose run --rm tools python scripts/e2e.py
```

Тест автоматически проверяет факт выявления всех 7 расчетных инцидентов сценария, соответствие планового и фактического индексов освоенного объема SPI, а также корректность формирования итогового PDF-отчета.

4.3. Точки доступа к работающей системе

После успешного развертывания веб-интерфейс и API доступны по адресам:

- **Пользовательский веб-интерфейс (React SPA):**
<http://localhost:8080/>
- **Интерактивная сводная спецификация API (Swagger UI):**
<http://localhost:8080/docs/>
- **Объектное S3-хранилище фотоматериалов и отчетов:**
<http://localhost:8080/storage/>
- **Прямая документация API микросервисов:**
 - Сервис календарного плана: <http://localhost:8080/api/v1/plan/docs>
 - Сервис строительной площадки: <http://localhost:8080/api/v1/site/docs>
 - Сервис аналитики и нарушений: <http://localhost:8080/api/v1/analysis/docs>
 - Сервис компьютерного зрения: <http://localhost:8080/api/v1/vision/docs>